

Theory of Computation

Lecture 1: Can Intelligent Machines Compute Everything?

Sheikh Azizul Hakim

Lecturer, Department of CSE, BUET

Last compiled: June 20, 2026 12:21am Z

This course is not about old machines.

It is about the limits of **all machines.**

Four Things That Look Easy

1. Parentheses

(((((((((((((())))))))))))))

2. JSON

```
{
  "students": [
    { "name": "Rahim",
      "courses": ["CSE", "EEE"] }
  ]
}
```

3. Program behavior

```
while True:
    pass
```

4. Sudoku

Given a filled board: check it.
Given an empty puzzle: solve it.

Four Things That Look Easy

1. Parentheses

((((((((((((((()))))))))))))))))

2. JSON

```
{
  "students": [
    { "name": "Rahim",
      "courses": ["CSE", "EEE"] }
  ]
}
```

3. Program behavior

```
while True:
    pass
```

4. Sudoku

Given a filled board: check it.
Given an empty puzzle: solve it.

These four examples motivate almost the entire course.

Prediction Game

Before learning the theory, guess the answer.

Problem	Solvable?	Efficient?
Check password rules	?	?
Check balanced parentheses	?	?
Validate JSON	?	?
Predict whether any program halts	?	?
Solve arbitrary Sudoku	?	?
Perfectly verify AI-generated code	?	?

Prediction Game

Before learning the theory, guess the answer.

Problem	Solvable?	Efficient?
Check password rules	?	?
Check balanced parentheses	?	?
Validate JSON	?	?
Predict whether any program halts	?	?
Solve arbitrary Sudoku	?	?
Perfectly verify AI-generated code	?	?

We will revisit this table throughout the semester.

The Four Mysteries

Mystery 1

Can finite memory understand arbitrarily deep structure?

Mystery 2

Can every problem be solved algorithmically?

Mystery 3

Can every program be analyzed automatically?

Mystery 4

Can every efficiently checkable solution also be efficiently found?

Mystery 1: Password Rules

A website asks for a password:

At least 8 characters

Contains a digit

Contains an uppercase letter

Contains no spaces

Mystery 1: Password Rules

A website asks for a password:

At least 8 characters

Contains a digit

Contains an uppercase letter

Contains no spaces

Abcdef12	valid
----------	-------

A1bcdefg	valid
----------	-------

abcdef12	invalid
----------	---------

ABC DEF12	invalid
-----------	---------

Mystery 1: Password Rules

A website asks for a password:

At least 8 characters

Contains a digit

Contains an uppercase letter

Contains no spaces

Abcdef12	valid
A1bcdefg	valid

abcdef12	invalid
ABC DEF12	invalid

How much memory is needed?

Mystery 1: Password Rules

A website asks for a password:

At least 8 characters

Contains a digit

Contains an uppercase letter

Contains no spaces

Abcdef12	valid
----------	-------

A1bcdefg	valid
----------	-------

abcdef12	invalid
----------	---------

ABC DEF12	invalid
-----------	---------

How much memory is needed?

Only finite memory. This leads to finite automata.

Mystery 1: Tokenizing Code

Your compiler sees this:

```
if (x == 10) return x + 1;
```

Mystery 1: Tokenizing Code

Your compiler sees this:

```
if (x == 10) return x + 1;
```

But internally it may see this:

```
IF LPAREN ID EQEQ NUM RPAREN RETURN ID PLUS NUM SEMICOLON
```

Mystery 1: Tokenizing Code

Your compiler sees this:

```
if (x == 10) return x + 1;
```

But internally it may see this:

```
IF LPAREN ID EQEQ NUM RPAREN RETURN ID PLUS NUM SEMICOLON
```

How does raw text become structure?

Mystery 1: Tokenizing Code

Your compiler sees this:

```
if (x == 10) return x + 1;
```

But internally it may see this:

```
IF LPAREN ID EQEQ NUM RPAREN RETURN ID PLUS NUM SEMICOLON
```

How does raw text become structure?

Regular expressions, DFA, and NFA are behind this story.

But Finite Memory Hits a Wall

Balanced:

()

(())

((()))

(() (()))

But Finite Memory Hits a Wall

Balanced:

```
()  
()  
()  
()  
()
```

Not balanced:

```
()()  
((  
())()
```

But Finite Memory Hits a Wall

Balanced:

```
()  
()  
()  
()  
()
```

Not balanced:

```
()  
()  
()  
()
```

Can finite memory handle arbitrary nesting?

But Finite Memory Hits a Wall

Balanced:

```
()  
()  
()  
()  
()
```

Not balanced:

```
()  
()  
()  
()
```

Can finite memory handle arbitrary nesting?

No. We need stronger models.

Mystery 2: Why VS Code Knows Something Is Missing

Incomplete JSON:

```
{  
  "students": [  
    {  
      "name": "Rahim",  
      "courses": ["CSE", "EEE"]  
    }  
  ]  
}
```

Mystery 2: Why VS Code Knows Something Is Missing

Incomplete JSON:

```
{  
  "students": [  
    {  
      "name": "Rahim",  
      "courses": ["CSE", "EEE"]  
    }  
  ]  
}
```

What is the editor remembering?

Mystery 2: Why VS Code Knows Something Is Missing

Incomplete JSON:

```
{  
  "students": [  
    {  
      "name": "Rahim",  
      "courses": ["CSE", "EEE"]  
    }  
  ]  
}
```

What is the editor remembering?

It remembers what still needs to be closed:

{ [{ [

Mystery 2: Why VS Code Knows Something Is Missing

Incomplete JSON:

```
{  
  "students": [  
    {  
      "name": "Rahim",  
      "courses": ["CSE", "EEE"]  
    }  
  ]  
}
```

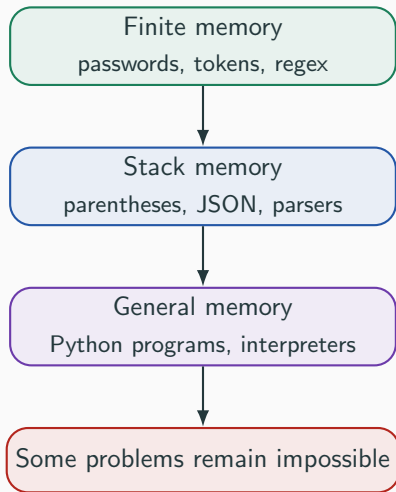
What is the editor remembering?

It remembers what still needs to be closed:

{ [{ [

This is stack-like memory. This leads to pushdown automata.

A Visual Hierarchy of Memory



Mystery 3: Will This Program Stop?

Obviously stops

```
print("Hello")
```

Obviously does not stop

```
while True:  
    pass
```

Mystery 3: Will This Program Stop?

Obviously stops

```
print("Hello")
```

Obviously does not stop

```
while True:  
    pass
```

But what about an arbitrary program with thousands of lines?

Mystery 3: Will This Program Stop?

Obviously stops

```
print("Hello")
```

Obviously does not stop

```
while True:  
    pass
```

But what about an arbitrary program with thousands of lines?

Can we build a perfect halt-detector?

Mystery 3: Will This Program Stop?

Obviously stops

```
print("Hello")
```

Obviously does not stop

```
while True:  
    pass
```

But what about an arbitrary program with thousands of lines?

Can we build a perfect halt-detector?

No. This is not an engineering failure. It is a theorem.

A Shocking Theorem Coming Later

Halting Problem

There is no program that can perfectly determine whether every other program eventually halts.

A Shocking Theorem Coming Later

Halting Problem

There is no program that can perfectly determine whether every other program eventually halts.

Consequences:

- no perfect bug detector,
- no perfect malware detector,
- no perfect program verifier for all programs,
- no AI system that can bypass this for arbitrary code.

A Shocking Theorem Coming Later

Halting Problem

There is no program that can perfectly determine whether every other program eventually halts.

Consequences:

- no perfect bug detector,
- no perfect malware detector,
- no perfect program verifier for all programs,
- no AI system that can bypass this for arbitrary code.

Useful tools exist. Perfect tools do not.

Mystery 4: Checking vs Finding

Checking

Given a completed Sudoku board, we can quickly check whether it is valid.

Finding

Given an empty or partial puzzle, finding a solution may require enormous search.

Mystery 4: Checking vs Finding

Checking

Given a completed Sudoku board, we can quickly check whether it is valid.

Finding

Given an empty or partial puzzle, finding a solution may require enormous search.

Why is verification often easier than discovery? Or is it?

Mystery 4: Checking vs Finding

Checking

Given a completed Sudoku board, we can quickly check whether it is valid.

Finding

Given an empty or partial puzzle, finding a solution may require enormous search.

Why is verification often easier than discovery? Or is it?

This question leads to P, NP, reductions, and NP-completeness.

AI Does Not Delete Complexity

An LLM may help us write a Sudoku solver.

AI Does Not Delete Complexity

An LLM may help us write a Sudoku solver.

But writing a solver is not the same as magically solving every instance instantly.

AI Does Not Delete Complexity

An LLM may help us write a Sudoku solver.

But writing a solver is not the same as magically solving every instance instantly.

AI can improve heuristics.

AI can guide search.

AI can generate code.

But AI does not erase worst-case complexity.

Can ChatGPT Prove Every Theorem?

First question:

Can ChatGPT prove every true mathematical theorem?

Can ChatGPT Prove Every Theorem?

First question:

Can ChatGPT prove every true mathematical theorem?

Second question:

Can it prove every theorem that has a short proof?

Can ChatGPT Prove Every Theorem?

First question:

Can ChatGPT prove every true mathematical theorem?

Second question:

Can it prove every theorem that has a short proof?

This connects theory of computation to:

- proof verification,
- automated theorem proving,
- Lean and formal mathematics,
- computational complexity.

Topic	Core question	Concrete examples
DFA/NFA	finite memory	passwords, tokens, regex
CFG/PDA	stack memory	JSON, parsers, parentheses
Turing machines	computation itself	interpreters, programs
Undecidability	impossible problems	halting, verification
Complexity	feasible computation	Sudoku, SAT, cryptography

Topic	Core question	Concrete examples
DFA/NFA	finite memory	passwords, tokens, regex
CFG/PDA	stack memory	JSON, parsers, parentheses
Turing machines	computation itself	interpreters, programs
Undecidability	impossible problems	halting, verification
Complexity	feasible computation	Sudoku, SAT, cryptography

The course is about the possible, the impossible, and the feasible.

How We Will Use AI in TCS/Mathematics

We will use AI systems as assistants, not oracles.

How We Will Use AI in TCS/Mathematics

We will use AI systems as assistants, not oracles.

- Ask an LLM to design an automaton.
- Test it with counterexamples.
- Ask for a proof.
- Find the hidden gap.
- Repair the reasoning formally.

How We Will Use AI in TCS/Mathematics

We will use AI systems as assistants, not oracles.

- Ask an LLM to design an automaton.
- Test it with counterexamples.
- Ask for a proof.
- Find the hidden gap.
- Repair the reasoning formally.

A good computer scientist does not merely get answers. A good computer scientist verifies them.

Michael Sipser

Introduction to the Theory of Computation

Michael Sipser

Introduction to the Theory of Computation

These slides are a modern companion to the classical theory.

By the End of This Course, You Should Know

- why regular expressions work,
- why JSON parsing needs memory,
- why some program-analysis tools can never be perfect,
- why AI cannot solve every search problem instantly,
- why P versus NP is one of the deepest questions in computing.

By the End of This Course, You Should Know

- why regular expressions work,
- why JSON parsing needs memory,
- why some program-analysis tools can never be perfect,
- why AI cannot solve every search problem instantly,
- why P versus NP is one of the deepest questions in computing.

Theory of Computation is the mathematics of computational limits.

Where Is Theory of Computation Alive Today?

Theory Meets Artificial Intelligence

Large Language Models

- Why do large models generalize?
- Why do they hallucinate?
- Can they reason systematically?
- What are their computational limits?

AI for Mathematics

- Automated theorem proving
- LLMs + Lean/Isabelle/Coq
- Formal verification of AI-generated code
- Machine-assisted scientific discovery

Can intelligence transcend the limits of computation?

Where Is Theory of Computation Alive Today?

Theory Meets Software and Systems

SAT and SMT Solvers

- Software verification
- Hardware verification
- Program synthesis
- Scheduling and planning
- Security analysis

Programming Languages

- Compilers
- Static analyzers
- Type systems
- Model checking
- Automated bug detection

Can every bug be detected automatically?

Where Is Theory of Computation Alive Today?

Theory Meets the Future of Computing

Complexity Theory

- P versus NP
- Fine-grained complexity
- Approximation algorithms
- Hardness of optimization
- Cryptographic assumptions

Emerging Paradigms

- Quantum computing
- Interactive proofs
- Zero-knowledge proofs
- Streaming algorithms
- Distributed computation

Which problems remain difficult even for future computers?

**If intelligence is computation,
what are the limits of intelligence itself?**